



DrishtiSec

Security Validation Report

Assessment Date	2026-09-13 13:00 UTC
Assessment Target	PresensiKu – Employee Attendance System
Environment	Isolated Lab (192.168.200.128:8080)
Assessment Type	Web Application Security Assessment

Executive Summary

This report documents a controlled black-box web application security assessment performed against an isolated laboratory target (PresensiKu). Three vulnerabilities – OS Command Injection, Local File Inclusion, and Directory Traversal – were independently discovered through application enumeration, endpoint discovery, and parameter testing, then validated for impact. Each validated finding was subsequently correlated against prior WAF analysis, which recorded matching request patterns as observed production traffic. All three findings were successfully validated.

This assessment was performed entirely within an isolated laboratory environment. It does not represent, and must not be interpreted as, exploitation of any production system.

Scope

Target application: PresensiKu – Employee Attendance System, running at 192.168.200.128:8080 inside an isolated laboratory network. Testing was limited to this application and host; no external systems were in scope or contacted.

Methodology

- Reconnaissance – identify the target application, technology stack, and normal functionality.
- Enumeration – discover reachable endpoints through direct browsing and content/directory enumeration.
- Application Discovery – map discovered endpoints to the application features and parameters that reach them.
- Manual Testing – probe discovered parameters for injection and path-handling weaknesses.
- Vulnerability Validation – confirm real, observable impact for each candidate weakness in the isolated lab.
- WAF Correlation – cross-reference each validated finding against prior WAF analysis to confirm the same pattern was observed in production traffic.
- Evidence & Reporting – capture request/response evidence and local artifacts, then compile this report.

Reconnaissance

Enumeration

Endpoints reachable on the target, as registered by the application at the time this report was generated:

Method	Endpoint
GET	/
GET, POST	/admin
GET, POST	/admin/config
GET	/api
GET	/api/file
GET	/download
GET, POST	/login
GET	/logout
GET	/portal
GET	/portal/cuti
GET	/portal/evidence
GET	/portal/findings
GET	/portal/perusahaan
GET	/portal/presensi
GET	/portal/presensi/checkin
GET	/portal/presensi/checkout
GET	/portal/profil
GET	/portal/reports
GET	/portal/reports/download
GET	/portal/reports/generate
GET	/portal/riwayat
GET	/portal/security
GET	/portal/settings
GET	/portal/validation-scenarios

GET	/portal/validation/<validation_id>
GET	/portal/waf-findings
GET	/read
GET, POST	/signup

The /admin path returned a distinct response from unregistered paths, indicating an internal panel; enumerating beneath it surfaced /admin/config.

Application Discovery

Endpoint	Parameter	How It Was Found
/admin/config	cmd	Content enumeration under /admin surfaced an internal ops panel; its "Configuration Diagnostic
/read	file	The /read endpoint was found during application enumeration; its "Lihat Dokumen" (view docum
/api/file	path	The /api/file endpoint and its "path" parameter were found through normal application use — th

Findings

VAL-CMD-001 — OS Command Injection (Severity: High)

The /admin/config endpoint passes the supplied cmd value directly into a shell command with no sanitization or allow-listing, allowing arbitrary operating-system command execution.

Business Impact: An attacker able to reach this endpoint could execute arbitrary commands with the privileges of the application process, potentially leading to full host compromise.

VAL-LFI-001 — Local File Inclusion (LFI) (Severity: High)

The /read endpoint joins the supplied file value onto its intended documents directory with no normalization or allow-listing, so "../" segments escape it and include the contents of arbitrary local files.

Business Impact: An attacker able to reach this endpoint could read local application files, potentially exposing configuration or credential material.

VAL-TRAV-001 — Directory Traversal (Severity: High)

The /api/file endpoint joins a supplied path value onto its own intended assets directory with no normalization, so "../" segments escape it and reach internal application/HR configuration outside the intended directory. This is the same underlying primitive as VAL-LFI-001, reached through a different application entry point and parameter name, and escaping to a different, distinct resource so the two findings' impact is clearly distinguishable.

Business Impact: An attacker able to reach this endpoint could read internal application and HR infrastructure configuration outside the intended public directory.

Validation Results

VAL-CMD-001 — OS Command Injection

Status: VALIDATED WAF Reference: SP_Asset-006 Endpoint: /admin/config

WAF request pattern: /admin/config?cmd%3Dcat/root/.aws/credentials

Local evidence: lab-data/root/.aws/credentials

WAF evidence: GET-based request pattern. Laboratory validation: GET and POST. POST was added as an additional controlled test of the same execution primitive and was NOT observed in the original WAF evidence.

VAL-LFI-001 — Local File Inclusion (LFI)

Status: VALIDATED WAF Reference: SP_Asset-011 Endpoint: /read

WAF request pattern: /read?file%3D../../../../.env

Local evidence: lab-data/.env

WAF evidence and laboratory validation both used GET.

VAL-TRAV-001 — Directory Traversal

Status: VALIDATED WAF Reference: SP_Asset-013 Endpoint: /api/file

WAF request pattern: /api/file?path%3D../../../../.env

Local evidence: lab-data/traversal-target/.env

WAF evidence and laboratory validation both used GET.

Evidence

VAL-CMD-001 — OS Command Injection

timestamp=2026-09-13T19:59:26+0700 source_ip=192.168.200.128 method=GET
endpoint=/admin/config status=EXECUTED

Operating-system command executed successfully

VAL-LFI-001 — Local File Inclusion (LFI)

timestamp=2026-09-13T19:59:41+0700 source_ip=192.168.200.128 method=GET endpoint=/read
status=ERROR

Execution/read failed

VAL-TRAV-001 — Directory Traversal

timestamp=2026-09-13T19:59:43+0700 source_ip=192.168.200.128 method=GET endpoint=/api/file
status=ERROR

Execution/read failed

Impact

Combined, these findings would allow an attacker to execute arbitrary operating-system commands and to read arbitrary files readable by the application process — including application configuration, database credentials, and cloud/service credentials. Any one of the three findings, if present in a production deployment, would be sufficient for significant data exposure; together they indicate a systemic lack of input validation across the application's file- and command-handling parameters.

Recommendations

- Never pass user-supplied input to a shell or subprocess call. If external command execution is required, use a fixed allow-list of commands and pass arguments without shell interpretation (e.g. shell=False with an argument list).
- Never build filesystem paths by concatenating user input. Resolve the requested path and verify it remains within the intended directory before opening it (canonicalize and prefix-check, or use a framework-provided safe-join helper).

- Apply the same path-handling fix consistently across every endpoint that accepts a file or path parameter (this assessment found the same class of issue on two independent endpoints).
- Do not expose internal/administrative endpoints without authentication and network-level restriction; remove or properly gate legacy admin panels before deployment.
- Add automated regression tests asserting that traversal sequences and shell metacharacters are rejected by these parameters.

WAF Correlation

Each finding below was discovered and validated independently, before its matching WAF reference was consulted. The WAF reference confirms that the same request pattern was previously observed as production traffic — it is corroborating evidence, not the source of discovery.

Validation	Discovery Source	WAF Reference	Status
VAL-CMD-001	Content enumeration under /admin surfaced an internal ops panel; its "Config" and "Diagnosis" links and page-source endpoints were not accessible.	SP-Asset-006	Validated
VAL-LFI-001	The /read endpoint was found during application enumeration; its "Lihat Dokumen" (View document) feature on the /api/file endpoint was not accessible.	SR-Asset-011	Validated
VAL-TRAV-001	The /api/file endpoint and its "path" parameter were found through normal application use — Valid Dokumen Perusa-	SP-Asset-013	Validated

Conclusion

All three findings — OS Command Injection, Local File Inclusion, and Directory Traversal — were independently discovered through application assessment and successfully validated with observable impact against the isolated laboratory target. Each finding's request pattern was subsequently corroborated by prior WAF analysis. This constitutes successful controlled reproduction in an isolated vulnerable environment, not exploitation of a production system.